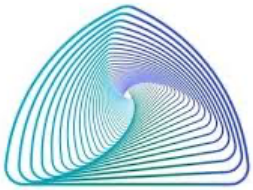


Scalable Real-time Detection of AI-Generated Arabic Text: A Distributed Data Pipeline Approach

Mohammed Alnajjar - 4714245

MSBDA-801

Supervised by:
Dr. Mohammed Alsarim

1. Abstract

The rapid growth of large language models has increased the need for reliable methods to distinguish between human-written and AI-generated text, especially in academic and Arabic-language contexts. This project developed a scalable Big Data pipeline for detecting AI-generated Arabic abstracts using Apache Spark, Spark DataFrames, Spark MLlib, and a simulated streaming workflow. The dataset used was KFUPM-JRCAI/arabic-generated-abstracts, which contains human-written Arabic abstracts and AI-generated versions from multiple models, including ALLAM, JAIS, LLaMA, and OpenAI. The original dataset contains 8,388 human abstracts, which were transformed into a binary classification format with human and AI-generated labels.

The pipeline included Arabic preprocessing, distributed storage in Parquet format, exploratory data analysis, traditional stylometric feature extraction, TF-IDF feature engineering, batch model training, and stream simulation. The traditional features used in the final feature-based model were Honoré's R measure, noun frequency, genitive construction count, and entity density. Three Spark MLlib classifiers were evaluated: Logistic Regression, Random Forest, and Linear SVM. Linear SVM achieved the best overall performance, with an accuracy of 0.6950, F1-score of 0.6942, and ROC-AUC of 0.7584. In the scalability benchmark, increasing Spark local cores from local[1] to local[4] reduced batch processing time from 2.67 seconds to 1.89 seconds. The streaming benchmark processed 100 records in 10 micro-batches with an average latency of approximately 0.095 seconds and an average throughput of about 109.60 records per second. These results show that the proposed pipeline can support scalable batch processing and near real-time AI-generated Arabic text detection.

2. Introdution

AI-generated text has become increasingly difficult to distinguish from human-written text due to the rapid development of large language models. These models can generate fluent, coherent, and domain-specific text, which creates challenges for academic integrity, publishing, digital trust, and content verification. In Arabic, this problem is especially important because AI-text detection resources and tools are less mature than those available for English.

The main objective of this project is to design and implement a distributed Big Data pipeline for detecting AI-generated Arabic text. The project follows a real-world Big Data workflow: data acquisition, distributed storage, Arabic preprocessing, scalable feature engineering, batch machine learning, stream simulation, model deployment, and performance benchmarking. The project description requires the use of distributed processing, Spark MLlib, efficient storage formats such as Parquet or ORC, and performance benchmarking based on scalability, throughput, and latency.

The project focuses on the following objectives:

1. Build a Spark-based pipeline for Arabic AI-generated text detection.
2. Store raw and processed datasets in efficient distributed formats.
3. Apply Arabic-specific preprocessing such as normalization, diacritic removal, tokenization, stop-word removal, and stemming/lemmatization.
4. Extract both traditional stylometric features and scalable text representations.
5. Train and evaluate multiple Spark MLlib classifiers.
6. Simulate real-time detection using micro-batch stream processing.

7. Compare batch and streaming performance using latency, throughput, and scalability benchmarks.

3. Related Work

AI-generated text detection has become an active research area because modern language models can produce text that closely resembles human writing. Recent surveys describe AI-generated text detection as a binary classification problem whose goal is to decide whether a given text was produced by a human or a language model. Wu et al. [1] review LLM-generated text detection methods and discuss major challenges such as out-of-distribution generalization, adversarial attacks, real-world evaluation problems, and the lack of standardized evaluation frameworks. Crothers et al. [2] also present a comprehensive survey of machine-generated text detection, emphasizing threat models, abuse scenarios, robustness, fairness, and accountability in detection systems.

Several detection methods rely on statistical or linguistic differences between human and machine-generated text. Earlier approaches used stylometric indicators, lexical richness, frequency distributions, function words, perplexity, burstiness, and supervised machine learning. These methods are useful because they can be interpretable and computationally lighter than deep learning models. However, they may become less reliable as LLMs improve and produce more human-like text. Deep learning-based detectors, such as RoBERTa-based GPT output detectors, learn contextual differences between human and generated text and are often effective when evaluated on data similar to their training distribution. Transformer-based detectors show the role of contextual language representations in identifying generated text [2].

DetectGPT is an important zero-shot method for detecting machine-generated text. Mitchell et al. [3] proposed that LLM-generated text tends to occupy negative curvature regions of a model's log probability function. DetectGPT uses probability curvature and random perturbations instead of training a separate classifier. Fast-DetectGPT later improved efficiency by replacing DetectGPT's perturbation step with a faster sampling-based method, making zero-shot detection more practical for larger-scale evaluation [4].

Other research investigates whether AI-generated text detection is theoretically possible. Chakraborty et al. [5] argue that detection is achievable when human and machine text distributions are not identical, but detection becomes harder as generated text becomes closer to human writing. Their work emphasizes that longer samples can improve detection reliability, which is important for abstract-level or document-level detection.

Watermarking is another major direction in AI-generated text detection. Kirchenbauer et al. [6] proposed a watermarking framework that embeds statistical signals into generated text during decoding, making generated text detectable later without significantly affecting quality. Later work by Kirchenbauer et al. [7] studied watermark reliability under realistic transformations such as paraphrasing, rewriting, and document mixing. Although watermarking is promising, it usually requires control over the generation process, while this project focuses on passive detection using extracted features and supervised classification.

From the Big Data architecture perspective, this project relates to Lambda and Kappa architectures. Lambda Architecture combines batch and streaming layers to support both historical analysis and low-latency processing. It is often used when systems require both accurate batch computation and real-time updates. Kappa Architecture simplifies this design by using a stream-first approach, where both real-time and historical processing can be handled through a single streaming pipeline. Feick et al. [8] compare Lambda and Kappa architectures for real-time data processing and discuss their trade-offs in scalable systems.

This project combines ideas from both areas: AI-generated text detection and scalable Big Data processing. The batch stage follows a Lambda-like design because historical data is processed offline for feature

engineering and model training, while the streaming stage simulates real-time inference on incoming Arabic abstracts.

4. Dataset Description

The dataset used in this project is KFUPM-JRCAI/arabic-generated-abstracts, which is available on Hugging Face. It contains Arabic academic abstracts and AI-generated versions of those abstracts. The dataset was selected because it directly supports the project goal of detecting AI-generated Arabic text.

The original dataset includes human-written Arabic abstracts and generated abstracts from several language models. The main text fields are:

- original_abstract: the original human-written Arabic abstract.
- allam_generated_abstract: the AI-generated abstract produced by ALLAM.
- jais_generated_abstract: the AI-generated abstract produced by JAIS.
- llama_generated_abstract: the AI-generated abstract produced by LLaMA.
- openai_generated_abstract: the AI-generated abstract produced by OpenAI.

The dataset is organized into three generation methods. The first method is by_polishing, which contains 2,851 samples. In this method, the model refines or polishes existing human abstracts. The second method is from_title, which contains 2,963 samples. In this method, the model generates an abstract from the paper title only. The third method is from_title_and_content, which contains 2,574 samples. In this method, the model generates an abstract using both the title and the paper content. The total number of original human abstracts across all generation methods is 8,388.

For this project, the dataset was converted into a binary classification format. The original_abstract column was labeled as human, while the generated abstract columns were labeled as ai_generated. Since each human abstract has multiple AI-generated versions, the final binary dataset contains more AI-generated samples than human samples.

The full binary dataset contains 41,940 text records. This includes 8,388 human-written texts and 33,552 AI-generated texts. Therefore, the dataset is imbalanced, with approximately 20% human-written texts and 80% AI-generated texts.

During initial data exploration, 5,427 duplicate text records were found. Duplicate records are important because they may cause data leakage if the same or very similar text appears in both the training and testing sets. For this reason, duplicate handling was considered before model training and evaluation.

For local development and experimentation, a smaller sampled dataset was used to reduce execution time and avoid memory issues.

The final processed dataset used for stylometric feature engineering contained the following columns:

- text: the Arabic abstract text.
- label: the class label, either human or ai_generated.
- honore_r: Honoré's R lexical richness measure.
- noun_frequency: frequency of nouns in the text.
- genitive_count: count of Arabic genitive constructions.
- entity_density: density of named entities in the text.

5. Methodology

A. System Environment and Pipeline Design

The project was implemented in one main notebook using PySpark and Spark MLlib. The local environment used Spark in local mode, with different resource configurations such as local[1], local[2], and local[4] for scalability testing. The pipeline followed these main stages:

1. Dataset loading
2. Initial exploration
3. Arabic preprocessing
4. Storage as Parquet
5. Stylometric feature extraction
6. TF-IDF feature engineering
7. Train/validation/test splitting
8. Batch model training
9. Model evaluation
10. Stream simulation
11. Scalability benchmarking

The project requirements include Spark-based preprocessing, Parquet/ORC storage, EDA, stylometric feature engineering, TF-IDF or word embeddings, Spark MLlib modeling, stream simulation, model deployment, and scalability testing.

B. Data Processing

The Arabic preprocessing pipeline was designed to clean and standardize the text before feature extraction and modeling. The preprocessing steps included:

Step	Description
Column renaming	Renamed problematic columns to text and label
Null handling	Removed rows with missing text or label values
Duplicate handling	Removed duplicate text records to reduce leakage risk
Arabic normalization	Normalized Arabic letter variants
Diacritic removal	Removed Arabic diacritics
Tatweel removal	Removed elongation character ـ
Symbol cleaning	Removed unwanted non-Arabic symbols
Whitespace cleaning	Collapsed repeated spaces
Stop-word removal	Removed common Arabic stop words
Stemming/lemmatization	Applied lightweight Arabic stemming in preprocessing stages

The processed dataset was saved in Parquet format for efficient reading and distributed processing.

C. Exploratory Data Analysis

EDA was conducted using Spark DataFrames. The analysis included:

- Dataset schema and data types.
- Total row count.
- Label distribution.
- Null value checking.
- Duplicate text count.
- Text length statistics.
- Word frequency analysis.
- Word cloud generation.
- N-gram frequency analysis.
- Vocabulary richness using Type-Token Ratio (TTR).

Word clouds and n-gram frequency outputs were generated from tokenized text. The n-gram analysis helped identify common repeated phrases in the human and AI-generated Arabic abstracts. TTR was used to measure lexical diversity by comparing the number of unique tokens with the total number of tokens.

D. Traditional Stylometric Feature Engineering

Four traditional stylometric features were engineered:

Feature	Description
honore_r	Measures lexical richness and vocabulary sophistication
noun_frequency	Ratio or frequency of nouns/proper nouns in the text
genitive_count	Count of likely Arabic genitive/idafa constructions
entity_density	Ratio of named entities to total words

Honoré's R measures vocabulary richness using total words, unique words, and words that appear once. Noun frequency and genitive construction features are linguistic indicators that capture grammatical structure. Entity density measures how frequently named entities occur in a text. These features were extracted as numerical columns in the Spark DataFrame.

The final stylometric feature dataset contained:

- text
- label
- honore_r
- noun_frequency
- genitive_count
- entity_density

E. Advanced Feature Engineering

The advanced scalable representation used was TF-IDF with Spark MLlib. The process included:

1. Tokenization using RegexTokenizer.
2. Arabic stop-word removal using StopWordsRemover.
3. Term-frequency vectorization using CountVectorizer.
4. Inverse-document-frequency weighting using IDF.

TF-IDF was used because it is scalable, interpretable, and suitable for traditional machine learning models in Spark. During some experiments, the TF-IDF vocabulary size was reduced for the sampled dataset to improve execution speed.

F. Train/Test Splitting and Data Leakage Prevention

The dataset was split into training, validation, and test sets. Duplicate text records were removed before splitting to reduce the risk of data leakage. This was important because duplicate samples could otherwise appear in both training and testing sets, leading to inflated evaluation results.

G. Batch Modeliong

Three Spark MLlib classifiers were trained and evaluated:

Model	Role
Logistic Regression	Baseline model
Random Forest	Advanced traditional classifier
Linear SVM	Advanced linear classifier

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion matrix

H. Stream Simulation and Real-Time Deployment

The streaming stage was implemented using manual micro-batch simulation because Spark Structured Streaming in the notebook introduced checkpoint and recovery issues. In the final stable implementation, Arabic abstracts were divided into small CSV batch files. Each file represented a newly arriving stream batch.

The stream simulation used the stylometric dataset containing the four extracted features. Each batch contained:

- text
- label
- honore_r
- noun_frequency
- genitive_count
- entity_density

For each micro-batch, the pipeline:

1. Loaded the CSV batch.
2. Assembled the four stylometric features into a Spark ML feature vector.
3. Applied the trained model.
4. Generated predicted labels.
5. Measured latency and throughput.

Latency and throughput were calculated as:

Latency = batch end time - batch start time

Throughput = number of records processed / latency

6. Results & Analysis

A. Model Performance Comparison

The final model comparison results are shown below.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.6755	0.6769	0.6755	0.6754	0.7560
Random Forest	0.6879	0.6900	0.6879	0.6877	0.7538
Linear SVM	0.6950	0.6997	0.6950	0.6942	0.7584

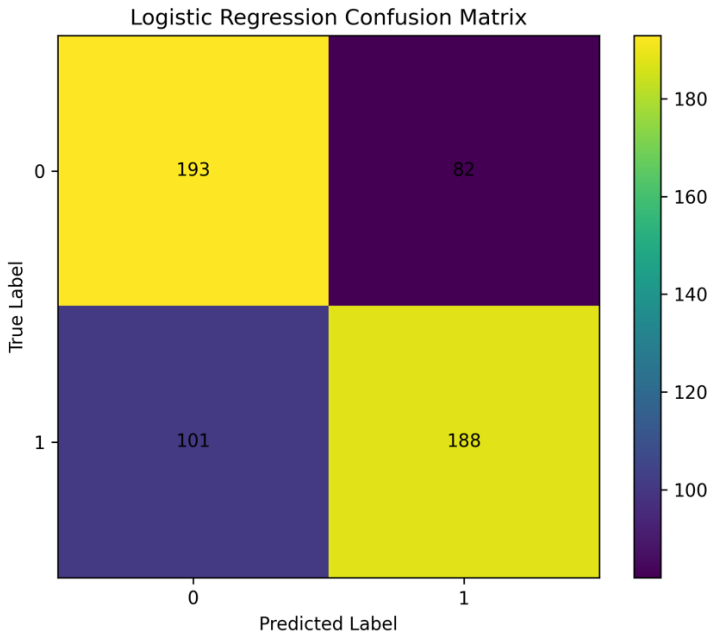
Linear SVM achieved the highest overall performance, with an accuracy of 0.6950, F1-score of 0.6942, and ROC-AUC of 0.7584. Random Forest achieved the second-best F1-score, while Logistic Regression provided a useful baseline but had the lowest overall performance among the three models.

The results suggest that Linear SVM performed slightly better for this feature representation. This is expected because SVM models are often effective for sparse or text-oriented classification tasks. Random Forest also performed competitively, likely because it can capture non-linear relationships between features. Logistic Regression remained useful as a fast and interpretable baseline.

B. Confusion Matrix Analysis

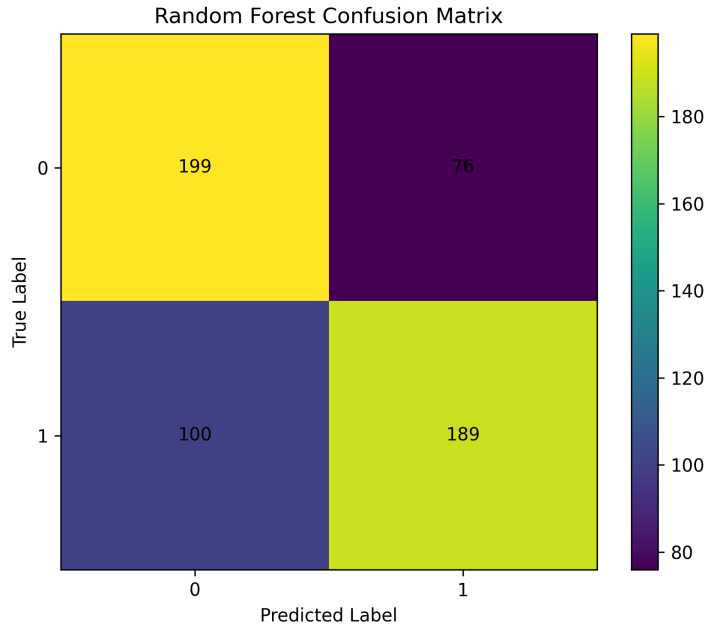
The confusion matrices are interpreted as follows.

Logistic Regression



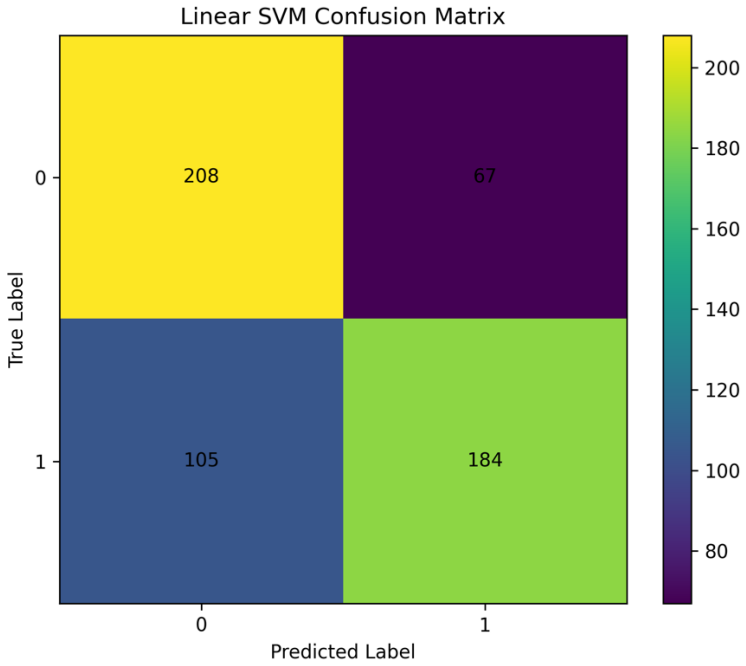
Logistic Regression correctly classified 193 AI-generated samples and 188 human samples. However, it misclassified 82 AI-generated samples as human and 101 human samples as AI-generated.

Random Forest



Random Forest improved the classification of class 0 compared with Logistic Regression, correctly identifying 199 AI-generated samples. It also correctly classified 189 human samples. It produced fewer AI-to-human errors than Logistic Regression.

Linear SVM



Linear SVM produced the highest number of correct class 0 predictions, correctly classifying 208 AI-generated samples and reducing class 0 misclassification to 67. However, it misclassified 105 class 1 samples as class 0. This shows that Linear SVM was strongest overall but still had class-level trade-offs.

C. Feature Coefficient Analysis

The feature coefficient analysis was performed on the traditional feature-based model.

Rank	Feature	Coefficient	Absolute Coefficient
1	noun_frequency	-15.7302	15.7302
2	entity_density	-12.9217	12.9217
3	genitive_count	0.0449	0.0449
4	honore_r	0.0002	0.0002

The most influential feature was **noun frequency**, followed by **entity density**. Both had large negative coefficients, meaning they strongly influenced the model toward the class encoded as 0 by the label indexer. The exact class interpretation depends on the label mapping, but based on absolute coefficient values, noun frequency and entity density were clearly the strongest predictors.

Genitive construction count had a much smaller effect, while Honoré's R measure had the weakest influence. This suggests that POS-based and named-entity-based linguistic signals were more useful than general lexical richness in this experiment.

D. Batch Scalability Benchmark

Batch scalability was tested using different Spark local resource settings.

Spark mode	Cores	Batch time (seconds)	Accuracy	F1-score	ROC-AUC
local[1]	1	2.6712	0.9792	0.9792	0.9966
local[2]	2	2.2059	0.9792	0.9792	0.9964
local[4]	4	1.8863	0.9792	0.9792	0.9964

The batch scalability benchmark showed that increasing the number of local Spark cores reduced processing time. The batch pipeline took 2.67 seconds with one core, **2.21 seconds** with two cores, and 1.89 seconds with four cores. This shows that the pipeline benefited from additional parallelism.

The evaluation metrics remained stable across the different resource configurations. This indicates that resource allocation affected processing time but did not meaningfully change model prediction quality.

E. Stream Processing Benchmark

The stream benchmark was conducted using 10 micro-batches, with each batch containing 10 records.

Metric	Result
Number of batches	10
Records per batch	10
Total streamed records	100
Average latency	0.0950 seconds
Minimum latency	0.0726 seconds
Maximum latency	0.1407 seconds
Average throughput	109.60 records/second
Minimum throughput	71.08 records/second
Maximum throughput	137.78 records/second

The streaming benchmark showed that the simulated real-time pipeline processed incoming batches efficiently. The average latency was approximately 0.095 **seconds per batch**, and average throughput was approximately 109.60 records per second. The highest throughput reached approximately 137.78 records per second.

These results indicate that the micro-batch stream simulation was lightweight and suitable for near real-time prediction on small incoming batches.

G. Batch vs. Stream Processing Trade-offs

Aspect	Batch Processing	Stream Processing
Main purpose	Training, tuning, full evaluation	Real-time prediction
Input	Stored dataset	Incoming micro-batches
Feature engineering	Can support heavier features	Should use lightweight or precomputed features
Main metrics	Accuracy, F1-score, ROC-AUC	Latency, throughput
Strength	Stronger evaluation and model comparison	Near real-time detection
Limitation	Not immediate	Sensitive to latency and feature consistency
Best use	Offline model development	Deployment and monitoring

Batch processing is better for training, feature engineering, tuning, and reliable evaluation. It can process historical data and support richer features such as POS-based stylometric indicators and entity density. However, it is not designed for immediate real-time prediction.

Stream processing is better for detecting AI-generated text as new records arrive. However, streaming introduces constraints such as latency, throughput, checkpointing, schema consistency, and feature compatibility. A key issue encountered during implementation was feature mismatch: the streaming model must receive the same feature vector size used during training. This was solved by using a stylometric stream dataset containing the same four features used by the model.

H. Error Analysis

The confusion matrices show that all models made errors in both directions. Some AI-generated texts were classified as human, and some human-written texts were classified as AI-generated. From a practical perspective, AI-generated texts classified as human are particularly important because they represent cases where AI-generated content passes detection. Human texts classified as AI-generated are also important because they may create false accusations in academic or professional settings.

The error patterns suggest that AI-generated and human-written Arabic abstracts share many linguistic similarities. This is expected because the dataset consists of academic abstracts, which often follow formal writing conventions regardless of whether they are human-written or AI-generated. In addition, the traditional stylometric features may not fully capture deeper semantic or discourse-level differences.

7. Conclusion & Future Work

This project implemented a scalable Big Data pipeline for detecting AI-generated Arabic text. The pipeline used Apache Spark for distributed data handling, Arabic preprocessing, Parquet storage, stylometric feature extraction, TF-IDF representation, Spark MLlib modeling, and stream simulation. The dataset was transformed into a binary classification task with `human` and `ai_generated` labels.

Three models were evaluated: Logistic Regression, Random Forest, and Linear SVM. Linear SVM achieved the best overall performance, with an accuracy of **0.6950**, F1-score of **0.6942**, and ROC-AUC of **0.7584**. Feature coefficient analysis showed that noun frequency and entity density were the most influential traditional stylometric features. Batch scalability testing showed that increasing Spark local cores reduced processing time, while stream simulation demonstrated low latency and strong throughput for small micro-batches.

The project also showed several important trade-offs. Batch processing is better for model training and full evaluation, while stream processing is more suitable for real-time prediction. However, stream deployment requires strict consistency between training features and streaming features. It also requires careful handling of schema, feature vector size, and latency.

I. Limitations

The project has several limitations:

1. A smaller sampled dataset was used for local experimentation due to hardware and execution-time constraints.
2. The streaming workflow was implemented as manual micro-batch simulation rather than a full Kafka-based production pipeline.
3. POS/NER-based features such as noun frequency and entity density can be computationally expensive for real-time processing.
4. The model performance remained moderate, suggesting that four stylometric features alone may not fully capture the complexity of AI-generated Arabic text.
5. The results may not generalize to Arabic dialects, social media text, news text, or non-academic writing styles.

II. Future Work

Future work can improve the project in several directions:

1. Use the full dataset for final training and evaluation on a more powerful cluster.
2. Integrate a full Kafka + Spark Structured Streaming pipeline.
3. Add transformer-based Arabic embeddings such as AraBERT or multilingual BERT.
4. Compare traditional ML models with deep learning and transformer classifiers.
5. Expand the feature set with additional Arabic linguistic, syntactic, semantic, and discourse features.
6. Evaluate the pipeline on other Arabic domains, including news, essays, social media, and dialectal Arabic.
7. Improve real-time feature extraction by precomputing expensive stylometric features or using optimized NLP services.
8. Perform more detailed class-level error analysis to understand why some AI-generated texts are misclassified as human and vice versa.

8. References

[1] J. Wu, S. Yang, R. Zhan, Y. Yuan, D. F. Wong, and L. S. Chao, “A Survey on LLM-Generated Text Detection: Necessity, Methods, and Future Directions,” *arXiv preprint arXiv:2310.14724*, 2023.

[2] E. Crothers, N. Japkowicz, and H. Viktor, “Machine Generated Text: A Comprehensive Survey of Threat Models and Detection Methods,” *arXiv preprint arXiv:2210.07321*, 2022.

[3] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, “DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature,” *arXiv preprint arXiv:2301.11305*, 2023.

[4] G. Bao, Y. Zhao, Z. Teng, L. Yang, and Y. Zhang, “Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature,” *arXiv preprint arXiv:2310.05130*, 2023.

[5] S. Chakraborty, A. S. Bedi, S. Zhu, B. An, D. Manocha, and F. Huang, “On the Possibilities of AI-Generated Text Detection,” *arXiv preprint arXiv:2304.04736*, 2023.

[6] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A Watermark for Large Language Models,” *arXiv preprint arXiv:2301.10226*, 2023.

[7] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, “On the Reliability of Watermarks for Large Language Models,” *arXiv preprint arXiv:2306.04634*, 2023.

[8] M. Feick, B. Kleer, and J. Kossmann, “Fundamentals of Real-Time Data Processing Architectures: Lambda and Kappa,” 2018.